Data Compression Techniques for Cache and Main Memory

SPARSH MITTAL

IIT ROORKEE, INDIA

Motivation

- Applications are becoming more data-intensive
- Memory requirement is increasing
- An example:
  - Exascale machine will require 100 PB data
  - 100PB DDR3 DRAM => 52MW power
  - More than 20MW budget of entire system!

Clearly, data compression techniques are essential for future systems

Benefits of Data Compression

- Higher effective capacity, without increasing memory size
- Avoids memory overflow: esp. useful in embedded systems
- Reduces miss-rate and bandwidth usage
- Space saved => turn-off that memory portion
- Saves energy => more computation in same power budget

Benefits of Data Compression

- Especially useful for NVMs and 3D memory:
  - Reduces amount of data written and write energy
  - Mitigates endurance problem in NVMs

- Storage compression (e.g., cache compression): reduces *data storage* energy
- Bandwidth (link) compression: reduces *data movement* energy

Opportunities for Compression

- Constants, data copying and assignment
- Initialization with common value
- Larger-sized data type for smaller data
- Large occurrence of special values, e.g., 0, 1

```
char source[] = "SourceText", dest[11];
  std::memcpy(dest, source, sizeof dest);

 long a[256], b[256], c[256];
 for(i = 0; i<256; i++) {
   b[i] = i;
   a[i] =0;
}
 for(i = 0; i<256; i++)
   c[i] =b[i];
```

Data copying

Large-size data type for smaller data

Large occurrence of zero

Assignment

Opportunities for Compression

- Many same pixels in an image

- Small difference between nearby values

Opportunities for Compression

Zero Values: initialization, sparse matrices, NULL pointers

0x00000000	0x00000000	0x00000000	0x00000000	…

Repeated Values: common initial values, adjacent pixels

0x*000000***FF**	0x*000000***FF**	0x*000000***FF**	0x*000000***FF**	…

Narrow Values: small values stored in a big data type

0x*00000***00**	0x*00000***0B**	0x*00000***03**	0x*00000***04**	…

Other Patterns: pointers to the same memory region

0x*C04039***C0**	0x*C04039***C8**	0x*C04039***D0**	0x*C04039***D8**	…

Using compression, data can be stored in smaller number of bits

Challenges in using Compression

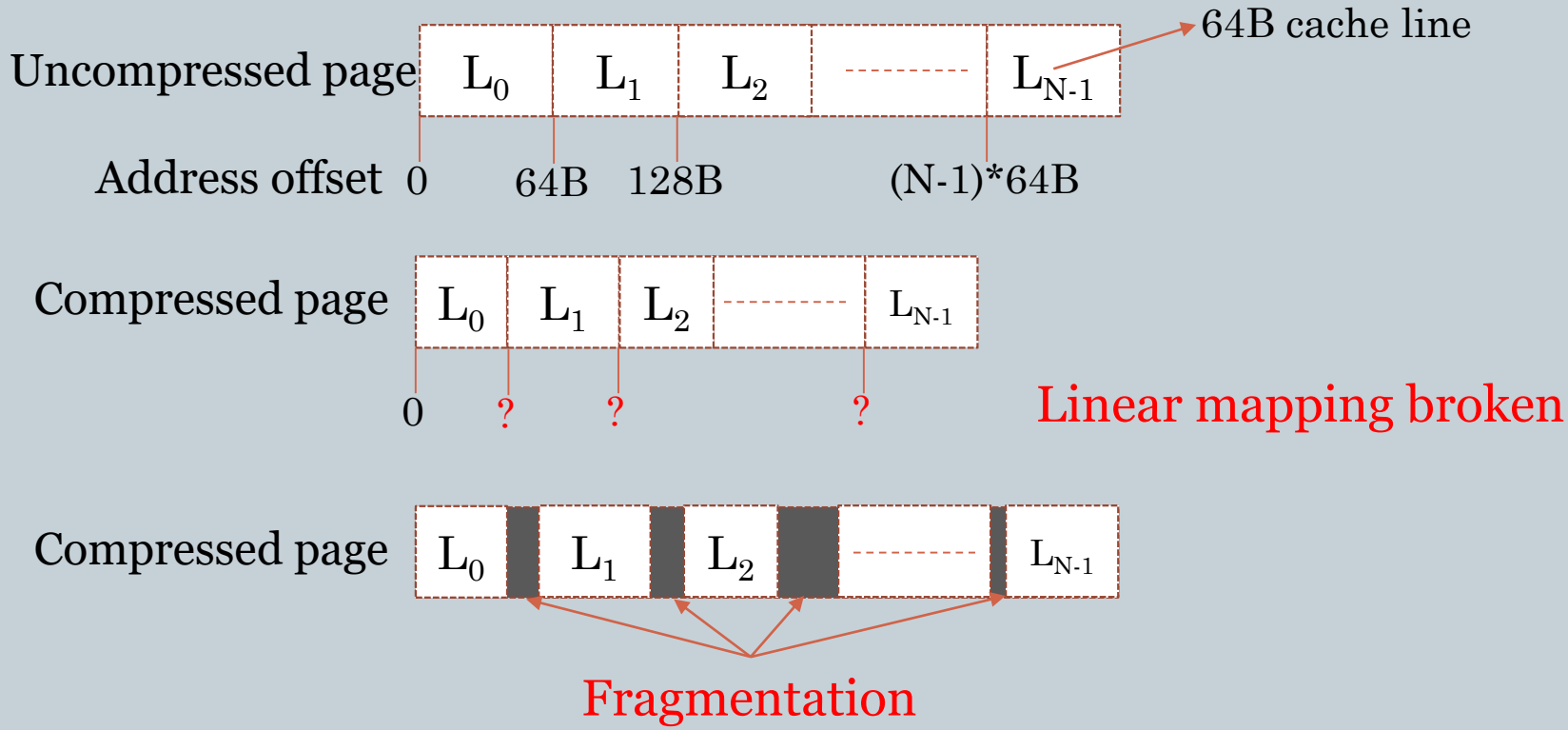
- Compression and decompression incur latency/energy
- Compression latency: not on critical path, but decompression latency: on critical path
- Compression algorithm providing high compressibility also have high area/latency overhead
- Useful in last level cache and main memory only: latency overhead not acceptable in L1 caches

Challenges in using Compression

- Variable length compression => parallel compression not possible
- Some techniques store incompressible lines separately ➔ higher overhead
- Incompressible data (e.g., encrypted) or inefficient compression algorithm => no benefit

Challenges in using Compression

- Compressed blocks have different size => linear mapping broken or fragmentation happens



Challenges in using Compression

- Size of evicted and inserted data may be different => replacement policy becomes complex
- Additional metadata
- If application memory footprint is already small => compression is of not much use
- If footprint is large => small additional capacity provided by compression not enough

Need of Carefully Choosing Data-Block Size

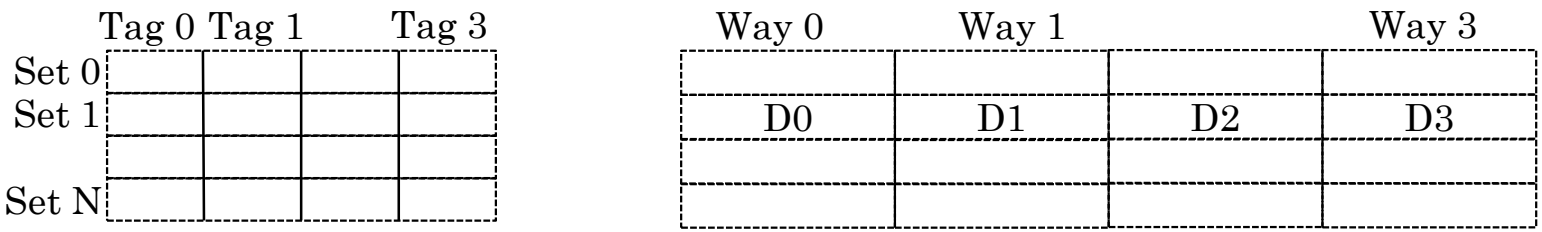
- The block-size at which compression is performed needs to be carefully chosen
- Higher block size =>
  - higher redundancy, lower metadata, higher compression ratio
  - Need to decompress whole block (e.g., 2KB) on access to even one sub-block (e.g., 64B)
  - Finding special patterns e.g., all-zero more difficult

Potential of compression

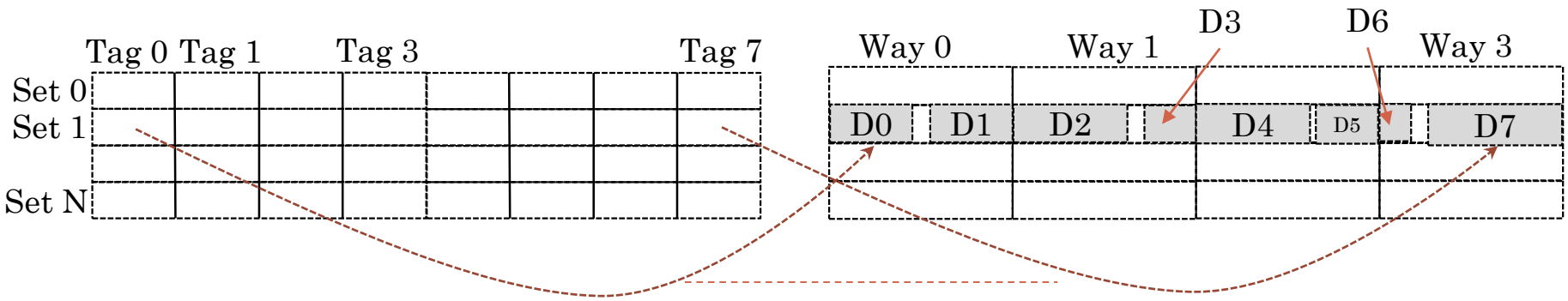
- If each unique block is stored just once, 16X compression ratio is possible [Arelakis et al. CAL'14]

- Practically: only 2-4X compression ratio is achieved since achieving 16X ratio would incur very high latency/storage overhead

Illustration of Cache Compression



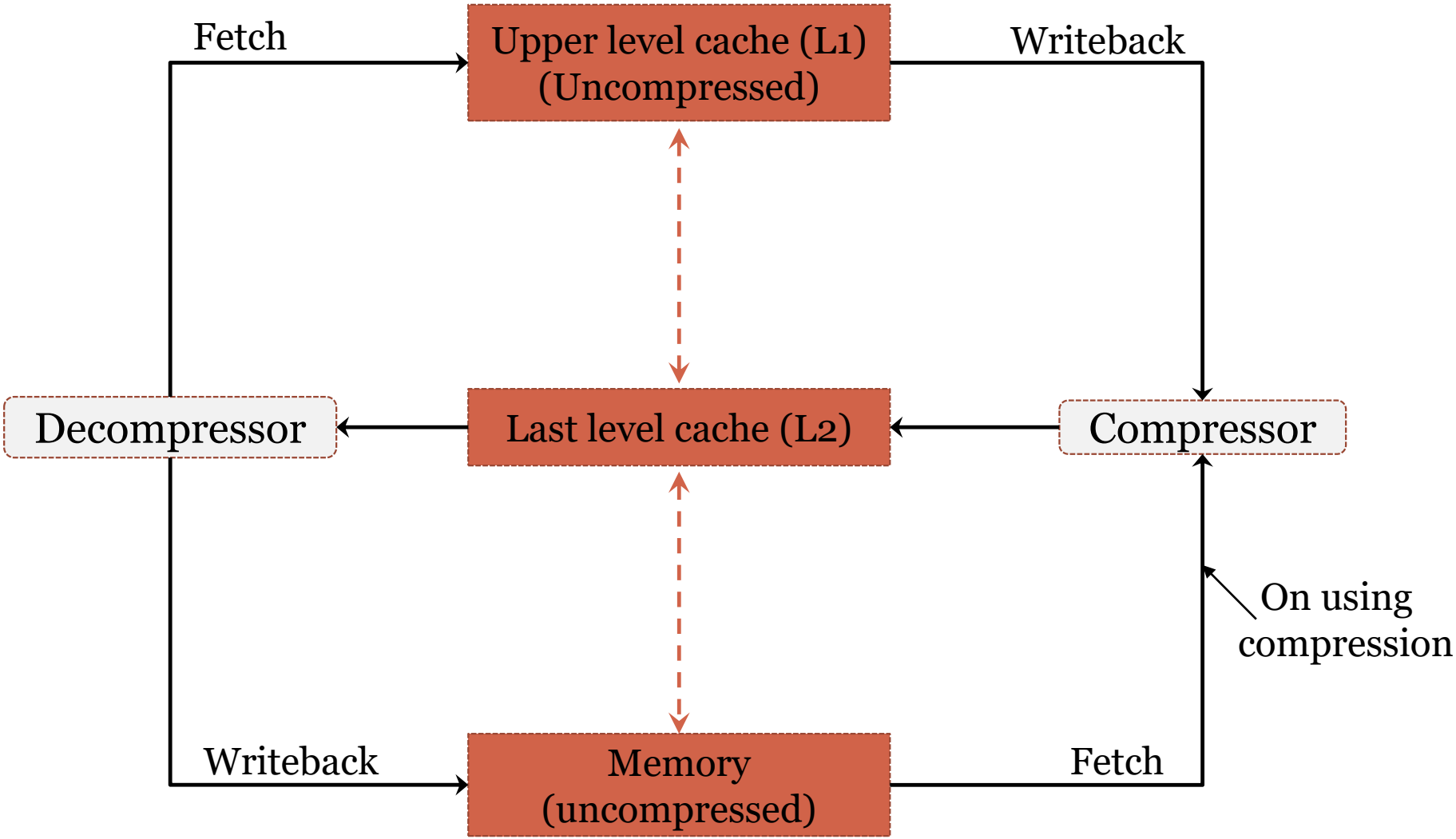
An uncompressed 4-way set-associative cache



A compressed cache with double tags to achieve up to 2X compression ratio

To achieve higher compression ratio, larger # of tags are required

Overall architecture with Cache Compression



On using no compression

On using compression

Some compression algorithms

- Huffman coding
- Lempel-Ziv (LZ) algorithm (and derivatives)
- X-match and X-RL
- Frequent value compression (FVC)
- Frequent pattern compression (FPC)
- C-PACK
- Base delta immediate (BDI) compression
- Zero-value and narrow-value detection
- .....

We will review these briefly

Working of Compression Algorithms

- **Huffman coding**
  - Find probability of different elements
  - Encode most probable elements with fewer number of bits

- **Lempel Ziv (LZ)**
  - Replace repeated occurrences of data elements with references to a single copy of that element existing earlier in uncompressed data.

Working of Compression Algorithms

- **X-match**
  - Use a dictionary.
  - Replace each input data element with a shorter code whenever a complete or partial match with a dictionary entry is found.

- **X-RL**
  - Before applying X-match, also encode length of consecutive zeros for improving compression efficiency

[Kjelso et al. EUROMICRO'96]

Working of Compression Algorithms

- **Frequent value compression** [Yang et al. MICRO'00]
  - Observation: Most accesses are directed to few hot values
  - Encode these frequent values to get compression
  - Frequent values become difficult to find with increasing cache block size

- **Frequent pattern compression** [Alameldeen et al. UWM TR'04]
  - Detect narrow words
  - Store them in compressed form with suitable prefix

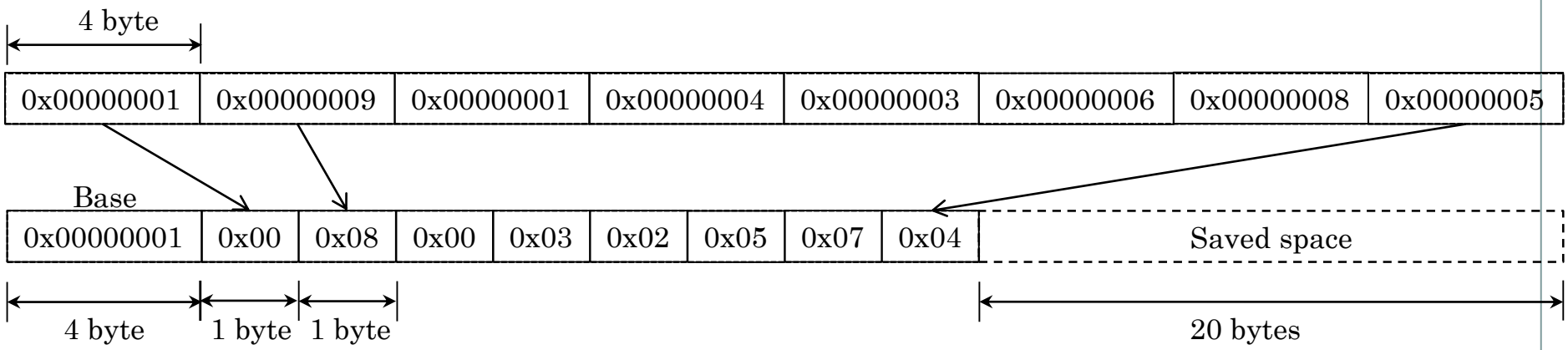
Working of **C-Pack** Compression Algorithm

- Pattern based partial dictionary match compression
- Compresses data by both statically and dynamically detecting frequently appearing data words
- Dictionary is dynamically updated ==> frequent words not captured by statically encoded patterns are captured
- Dictionary supports both partial and full word-matching
- A compressed line can be placed in same line with a partner if total size <= uncompressed line size

Working of **BDI** Compression

- For many cache lines, differences between values stored within the cache words are small.
- Represent a cache line using a base value and an array of differences, such that total size < original cache line size.
- Use two bases of which one is always zero. This helps in compressing both narrow and wide values.
- If any delta exceeds a fixed-size, cache line is considered incompressible

An example of BDI compression



A Comparison

	Characteristics			Compressible Data Patterns			
	Decompr. Latency	Complexity	Compr. Ratio	Zeros	Repeated Value	Narrow	Low Dynamic Range
Zero detection	Low	Low	Low	Y	N	N	N
FVC	High	High	Modest	Y	Partly	N	N
FPC	High	High	High	Y	Y	Y	N
BDI	Low	Modest	High	Y	Y	Y	Y

Decompr. = decompression, compr. = compression

Pekhimenko et al. PACT'12

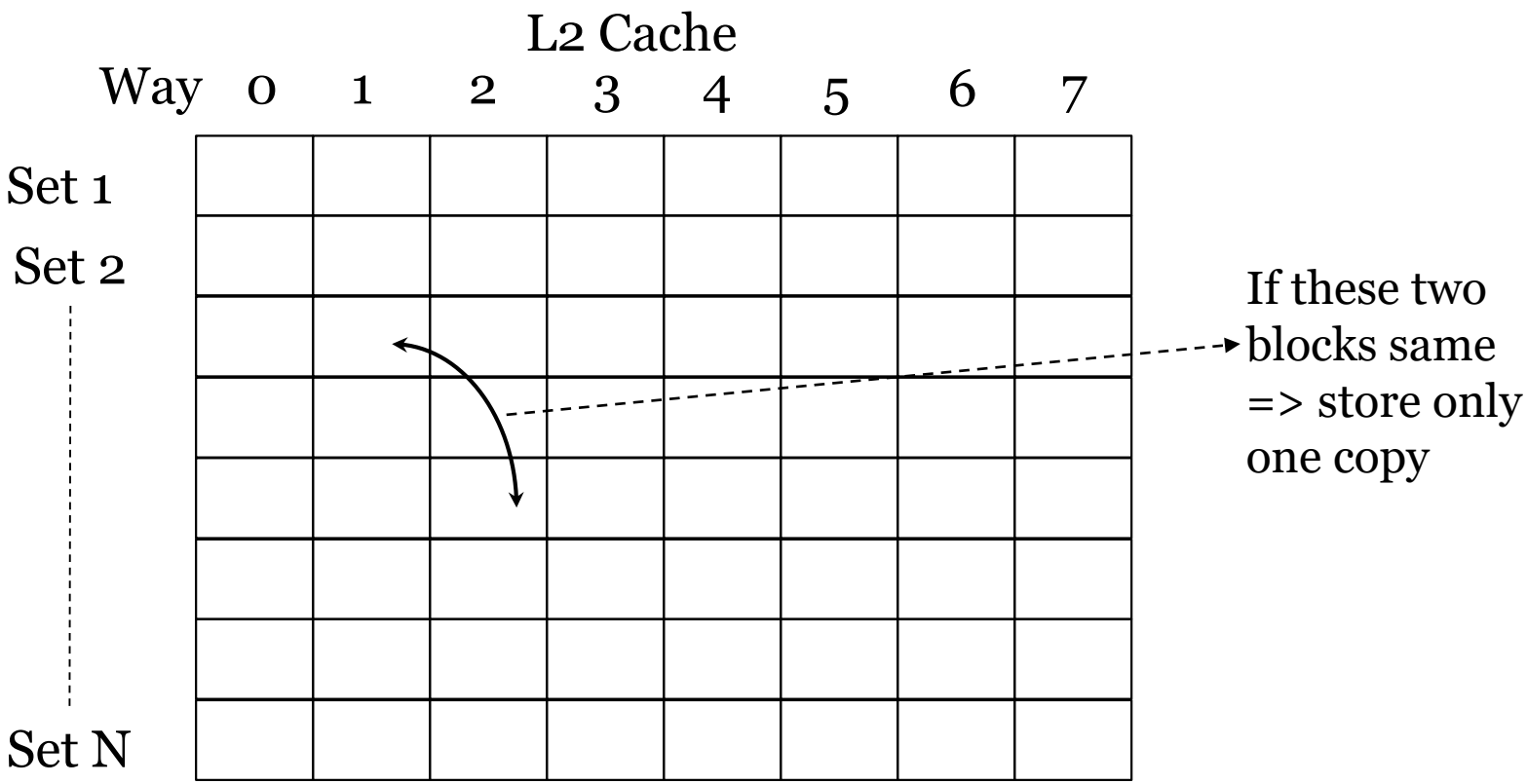
Granularity of Exploiting Redundancy

- Across *different blocks of the whole cache* (also called deduplication)
- Across *different words of a cache block*
- Across *different bytes of a cache word*.

We will now see examples of these.

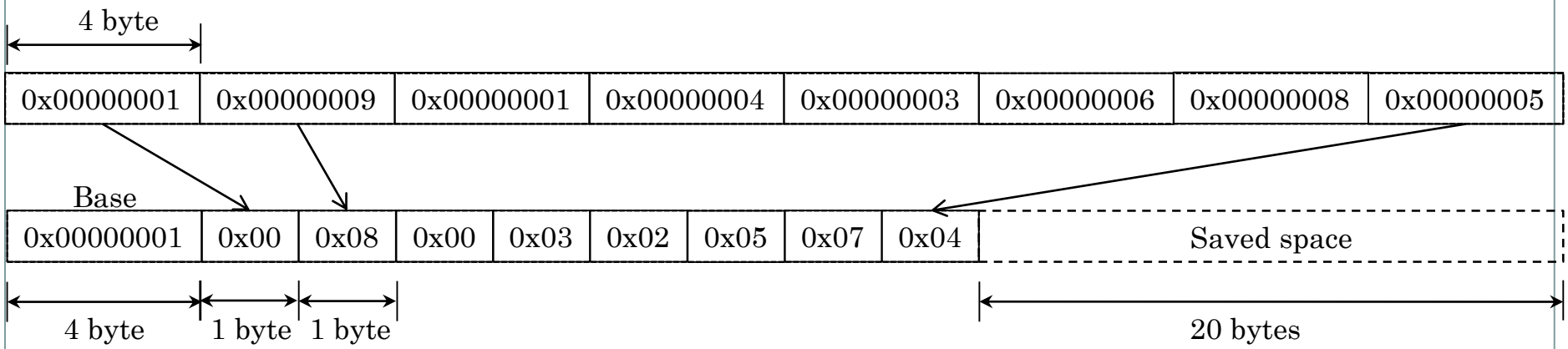
Granularity of Exploiting Redundancy

- Across *different blocks of the whole cache*



Granularity of Exploiting Redundancy

- Across *different words of a cache block*



- Across *different bytes of a cache word.*

Narrow Values: small values stored in a big data type

0x*00000*00 | 0x*000000*0B | 0x*00000*03 | 0x*00000*04 | …

Adaptive Compression Techniques

- Given the challenges of compression, using it for all blocks/cases is not best.
- Hence, many techniques use it adaptively, e.g.:
- Compress inactive or least important apps in background to free memory
- Not compressing hot blocks
- Divide memory in compressed/uncompressed portions. Perform compression only in compressed portion

Adaptive Compression Techniques

- Use counters to predict whether compression would have avoided a miss or improved performance. If no, don't compress that block
- Identify program phases with high spatial locality and perform compression only in these phases
- Compress only those blocks providing higher than a threshold compression ratio

Adaptive Compression Techniques

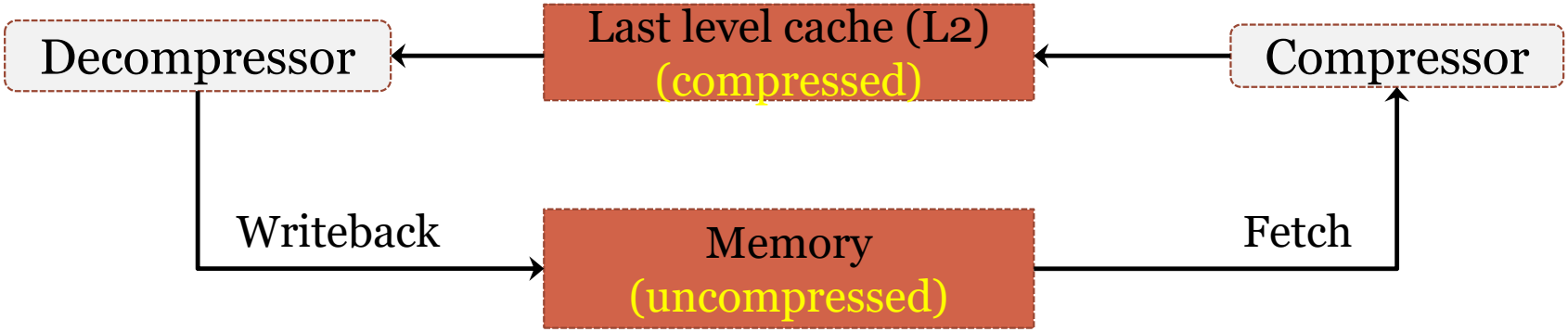
- Different compression algorithms for different data types/patterns, e.g., :
  - BDI for low-dynamic range
  - Zero-detection for zero data
  - FPC when frequent values are narrow or zero
  - Others for floating point data

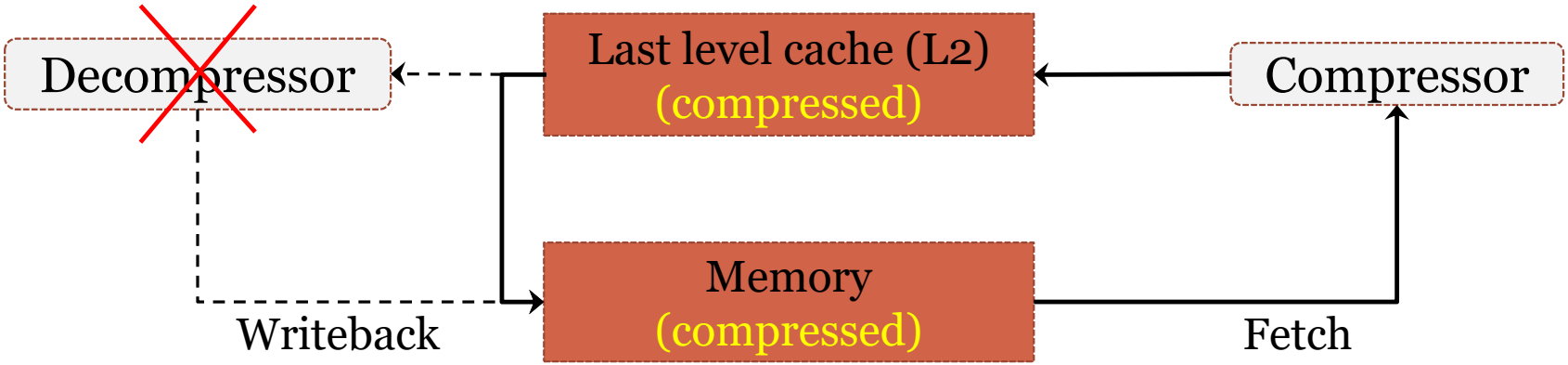
- Predict data type and use the most-suitable algorithm for this

Reducing overhead of Compression

- For deduplication, allow multiple tags to be associated with a data block. Then, keep only one unique copy of a block that can be accessed through multiple addresses
- To avoid fragmentation, perform compaction "in background"
- Use simpler compression schemes, e.g., Zero-compression ➔ decompression circuit trivial

Compression in cache v/s in both cache & main memory





Decompression not required for transmitting data

Hiding/Avoiding Decompression latency

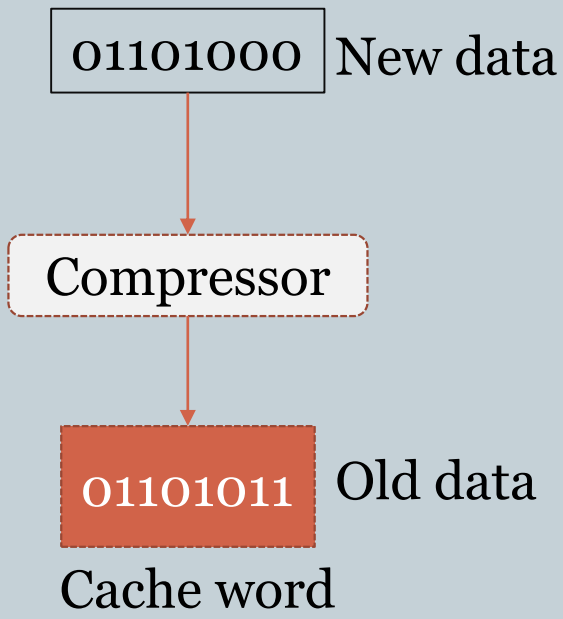
- Buffer recently decompressed data to hide decompression latency
- Predict blocks expected to be accessed in future. Decompress them beforehand to avoid latency
- First decompress the critical word
- Overlap data transfer time with decompression time

Optimizations enabled by Compression

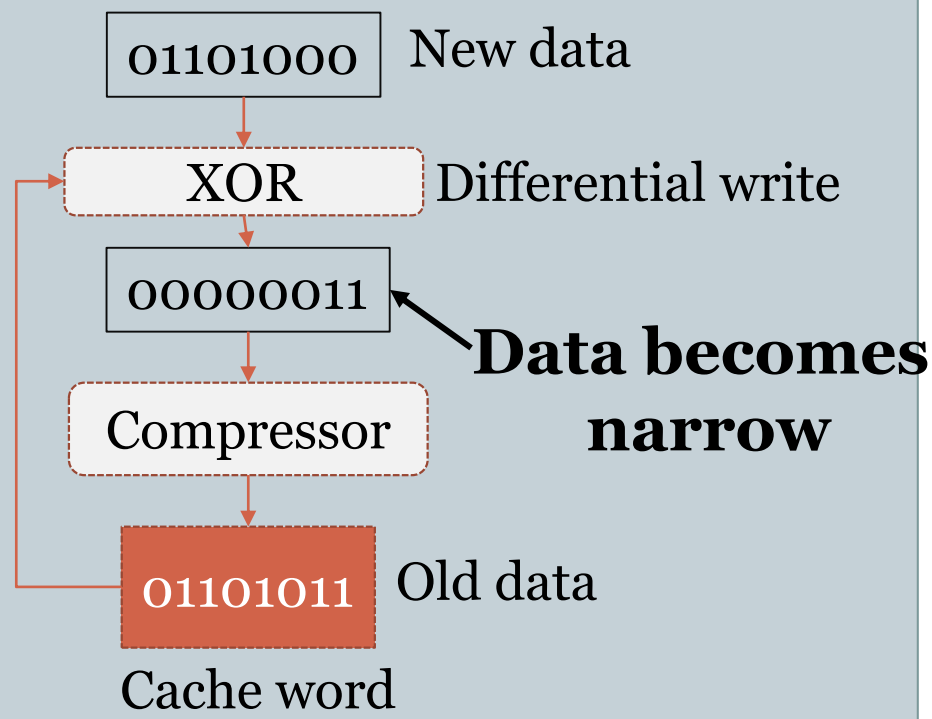
- Bandwidth saved by compression: use it to prefetch more blocks
- High compressibility => high predictability of data. Based on this, predict miss pattern and improve prefetching
- Use lossy compression with approximate computing approach

Improving data compressibility

- Use differential write before compression



Compression without Differential write



Compression after Differential write

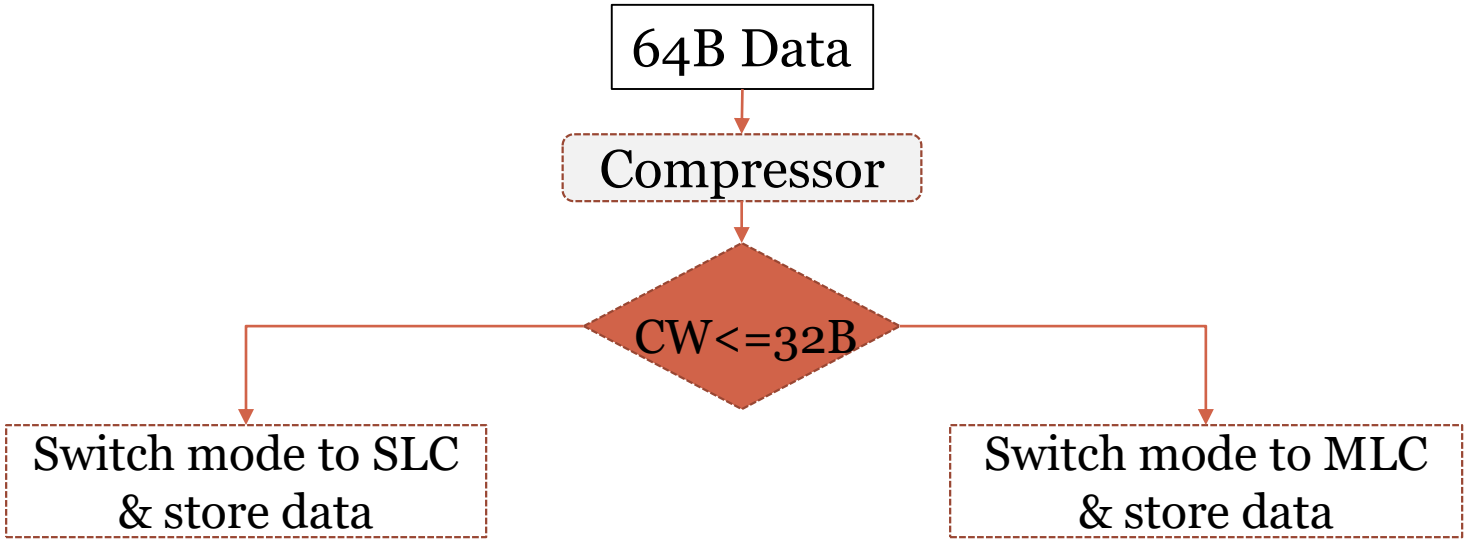
Du et al. TACO'13

Leveraging compression in PCM

- MLC and SLC PCM offer different features

SLC (single-level cell)	(+) Lower latency, higher endurance (-) density half of MLC (one bit per cell)
MLC (multi-level cell)	(+) higher density (two bits per cell) (-) Much longer latency, much lower endurance

- Choose SLC or MLC based on compressed width (CW)



PCM = phase change memory

Compression in GPU Register file (RF)

- All threads in a warp execute the same instruction => RF access happens at warp granularity.
- Threads in a warp are identified using thread-indices => computations that depend on thread-indices operate on register data that show strong value similarity.
- => Within a warp, the arithmetic difference between two consecutive thread registers is small
- Perform BDI on different thread values

Compression in GPUs

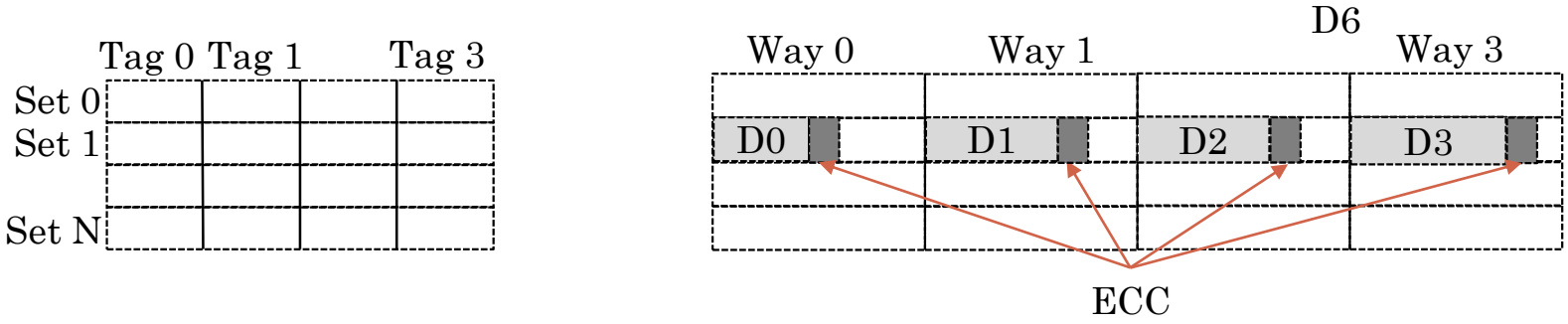
- Perform compression in bandwidth-limited workloads only and not in compute-resource limited workloads
- Generate assist warps which perform compression and decompression in parallel

Compression for other optimizations

- Generally, compression is used for gaining capacity by cramming more than one block in a cache line
- Even without this, compression is useful.
- Save only one block in a cache line => no fragmentation, reduced complexity

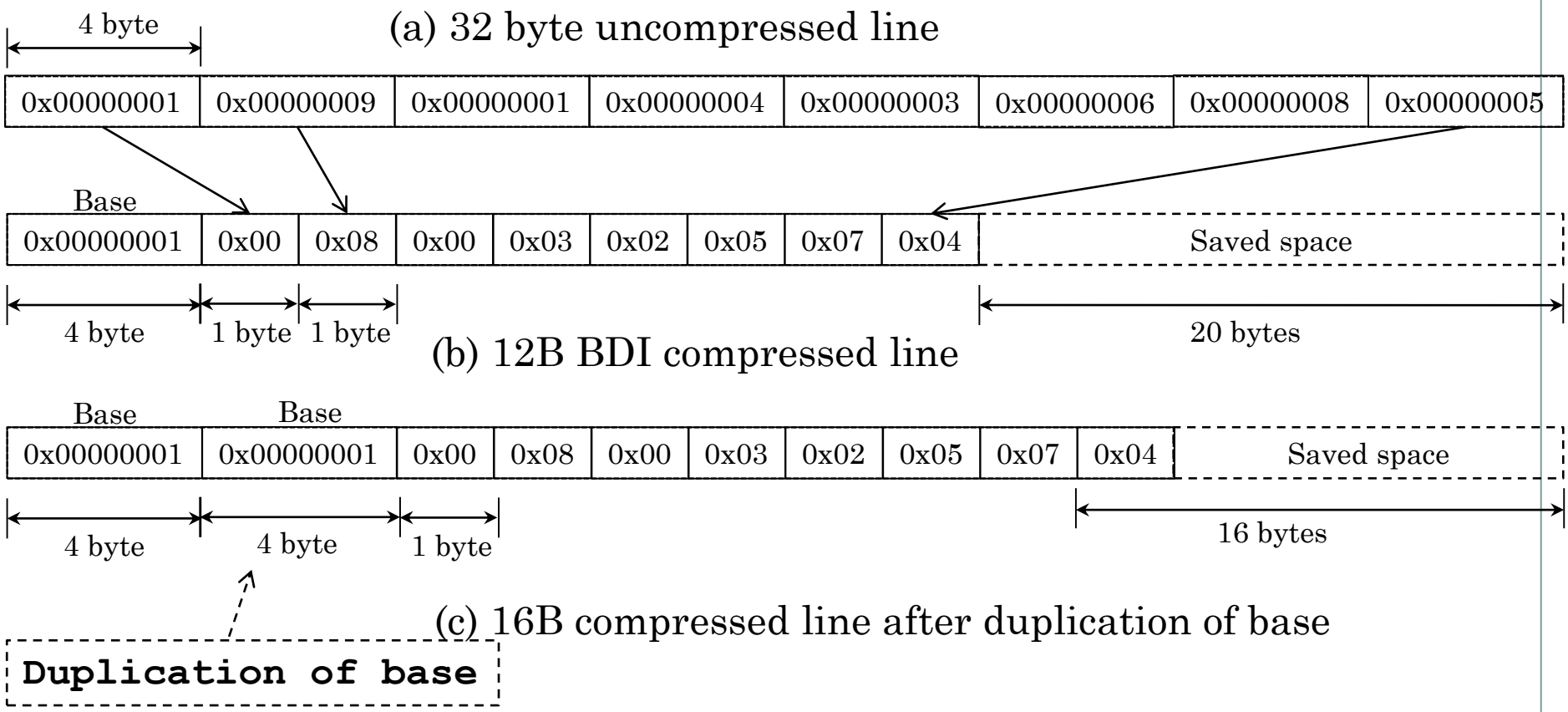
- Space saved by compression can be:
  - Used for ECC
  - Used for duplicating data to improve reliability
  - Power-gated to save leakage energy

Compression for Improving Reliability



Using space saved by compression for storing ECC

Providing higher protection to base by duplication



Compression in product systems

- Zswap: virtual memory compression approach in Linux kernel
- Active memory expansion (AME): a compression technique in IBM POWER7
- "Compressed memory" approach in Apple OS X

References

- Sparsh Mittal et al., "A Survey Of Architectural Approaches for Data Compression in Cache and Main Memory Systems", IEEE TPDS, 2016. (pdf)